HW 0 - Part 2: Language Model (LM) Fine-tuning with Huggingface

In this assignment, you will implement Pytorch code to train a language model (LM) using the 🤗 Transformers library. You will fine-tune a pre-trained GPT-2 model on a Harry Potter corpus, and evaluate the model on a lauguage modeling task (a.k.a. next token prediction). If you are familiar with 🤗 Transformers and 🤗 Datasets, feel free to skip steps 0 through 2.

Step 0: Installation

If you are using Google Colab or a fresh Python environment, you will need to install the required libraries:

Uncomment and run the following cell to install 🤗 Transformers and 🤗 Datasets:

```
#! pip install transformers datasets
```

- `transformers` : Provides pre-trained models like GPT-2 for fine-tuning.
- `datasets` : Offers easy access to datasets.

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "0"
```

Step 1: Preparing the dataset

We will use the Harry Potter corpus dataset to fine-tune the GPT-2. The 🤗 Datasets library makes it simple to load datasets.

Run the following code to load the dataset using `load_dataset` :

```
from datasets import load_dataset

# Load the Harry Potter corpus dataset
datasets = load_dataset('WutYee/HarryPotter_books_1to7')

# Preview the dataset structure
print(datasets)
```

```
/home/mrinaal/git/cse291a/hw0/.venv/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.
  from .autonotebook import tqdm as notebook_tqdm
DatasetDict({
    train: Dataset({
        features: ['text'],
        num_rows: 81349
    })
    validation: Dataset({
        features: ['text'],
        num_rows: 23118
    })
    test: Dataset({
        features: ['text'],
        num_rows: 23620
    })
})
```

As shown in `DatasetDict` , the dataset is typically split into subsets like `train` , `test` , or `validation` . To access a specific example, you must choose a split and an index.

```
# Access an example from the 'train' split
example = datasets["train"][10]

# Print the example
print(example)
```

```
{'text': 'by J.K. Rowling'}
```

Step 2: Preprocessing the Dataset

To fine-tune GPT-2, we need to tokenize the dataset text into a format the model can process. To tokenize all our texts with the same vocabulary that was used when training the model, we have to download a pretrained tokenizer. This is all done by the `AutoTokenizer` class:

```
from transformers import AutoTokenizer

model_name = 'gpt2'
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

The tokenizer will split the text into tokens and convert them to numerical IDs.

We can now call the tokenizer on all our texts. This is very simple, using the `map` method from the Datasets library. First we define a function that call the tokenizer on our texts:

```
def tokenize_function(examples):
    return tokenizer(examples["text"])
```

Then we apply it to all the splits in our `datasets` object, using `batched=True` and 4 processes to speed up the preprocessing. We won't need the `text` column afterward, so we discard it.

```
# Apply the tokenizer to the dataset
tokenized_datasets = datasets.map(tokenize_function, batched=True, num_proc=4, remove_columns=["text"])
```

If we now look at an element of our datasets, we will see the text have been replaced by the `input_ids` the model will need:

```
tokenized_datasets["train"][1]
```

```
{'input_ids': [50, 8387, 11751, 338, 8026], 'attention_mask': [1, 1, 1, 1, 1]}
```

Now for the harder part: we need to concatenate all our texts together then split the result in small chunks of a certain `block_size`. To do this, we will use the `map` method again, with the option `batched=True`. This option actually lets us change the number of examples in the datasets by returning a different number of examples than we got. This way, we can create our new samples from a batch of examples.

First, we grab the maximum length our model was pretrained with. This might be a big too big to fit in your GPU RAM, so here we take a bit less at just 128.

```
block_size = 128
```

Then we write the preprocessing function that will group our texts:

```
def group_texts(examples):
    # Concatenate all texts.
    concatenated_examples = {k: sum(examples[k], []) for k in examples.keys()}
    total_length = len(concatenated_examples[list(examples.keys())[0]])
```

```
    # We drop the small remainder, we could add padding if the model supported it
    # instead of this drop, you can customize this part to your needs.
    total_length = (total_length // block_size) * block_size
    # Split by chunks of max_len.
    result = {
        k: [t[i : i + block_size] for i in range(0, total_length, block_size)]
        for k, t in concatenated_examples.items()
    }
    result["labels"] = result["input_ids"].copy()
    return result
```

First note that we duplicate the inputs for our labels. This is because the model of the 🤗 Transformers library apply the shifting to the right, so we don't need to do it manually.

Also note that by default, the `map` method will send a batch of 1,000 examples to be treated by the preprocessing function. So here, we will drop the remainder to make the concatenated tokenized texts a multiple of `block_size` every 1,000 examples. You can adjust this behavior by passing a higher batch size (which will also be processed slower). You can also speed-up the preprocessing by using multiprocessing:

```
lm_datasets = tokenized_datasets.map(
    group_texts,
    batched=True,
    batch_size=1000,
    num_proc=4,
)
```

And we can check our datasets have changed: now the samples contain chunks of `block_size` contiguous tokens, potentially spanning over several of our original texts.

```
tokenizer.decode(lm_datasets["train"][1]["input_ids"])
```

```
'�t hold with such nonsense.      Mr. Dursley was the director of a firm called Grunnings, which madedrills. He was a big, beefy man with hardly any neck, although he did have avery large mustache. Mrs. Dursley was thin and blonde and had nearly twice theusual amount of neck, which came in very useful as she spent so much of hertime craning over garden fences, spying on the neighbors. The Dursleys had asmall son called Dudley and in their opinion there was no finer boy anywhere.      The Dursleys'
```

Now that the data has been cleaned, we're ready to train our model. 🤗 Transformers provides APIs and tools to easily download and train pretrained LM models. First we load the pre-trained GPT-2 model using `AutoModelForCausalLM.from_pretrained`.

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(model_name)
```

Now we need to implement fine-tuning for a pre-trained GPT-2 model and evaluate the model on a language modeling task.

Step 3: Fine-Tuning the GPT-2 Model.

1. **Implement the Training Loop and Perplexity Evaluation**:
    - Write a training loop to fine-tune the GPT-2 model
    - You may experiment with different optimizers, learning rates, and batch sizes. Here are the default values to start with: - Learning rate: 2e-5 - Optimzer: AdamW - Batch size: 8
    - Include an evaluation function to calculate **perplexity** on the validation set at the end of each epoch.
    - You may refer to open-source trainer implementations such as miniGPT for guidance.
2. **Validation and Test Evaluation**:
    - After each epoch, evaluate your model on the **validation set** and record the perplexity.

- Once training is complete (after 3 epochs), evaluate the final model on the **test set**.

Your goal is to achieve a **perplexity** in the range of **30–50** after **3 epochs** of training.

To receive full credit, you must report the following:

- Training loss and perplexity on the **validation set** for each of the 3 epochs.
- The final perplexity score on the **test set**.

e.g.,

Example Output

Epoch	Training Loss	Perplexity on Validation Set
1	3.14	18.37
2	2.98	17.83
3	2.91	17.73

Final Perplexity on the Test Set: 43.46

```
import torch
from torch.utils.data import DataLoader
from torch.optim import AdamW
from transformers import get_scheduler
from torch.optim.lr_scheduler import LinearLR
import numpy as np
```

```
def calculate_perplexity(loss):
    return np.exp(loss)
```

```
train_dataset = lm_datasets["train"]
val_dataset = lm_datasets["validation"]
test_dataset = lm_datasets["test"]

# Without this, the batch["key"] was a list of tensors, each list of length `block_size` and each tensor of size `batch_size`
train_dataset.set_format(type='torch', columns=['input_ids', 'attention_mask', 'labels'])
val_dataset.set_format(type='torch', columns=['input_ids', 'attention_mask', 'labels'])
test_dataset.set_format(type='torch', columns=['input_ids', 'attention_mask', 'labels'])
```

```
def train_step(
        model,
        train_dataloader,
        optimizer,
        lr_scheduler,
        device="cpu",
) -> float:
    model.train()
    total_train_loss = 0
    loss_list = []
    num_batches = len(train_dataloader)

    for bi, batch in enumerate(train_dataloader):
        batch = {
            key: torch.tensor(value).to(device)
            if not isinstance(value, torch.Tensor) else value.to(device)
            for key, value in batch.items()
        }

        # Forward pass
        outputs = model(**batch)
        loss = outputs.loss

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
```

```
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        if lr_scheduler:
            lr_scheduler.step()

        # Update the progress bar
        total_train_loss += loss.item()
        loss_list.append(loss.item())
        if (bi + 1) % 100 == 0:
            print(f"Progress [{bi+1:4d} / {num_batches}]: Loss: {total_train_loss / (bi + 1):.6f}")

    return total_train_loss, loss_list

def evaluate(
        model,
        val_dataloader,
        device: str = "cpu",
) -> float:
    model.eval()
    total_val_loss = 0
    loss_list = []
    num_batches = len(val_dataloader)

    with torch.no_grad():
        for bi, batch in enumerate(val_dataloader):
            batch = {
                key: torch.tensor(value).to(device)
                if not isinstance(value, torch.Tensor) else value.to(device)
                for key, value in batch.items()
            }

            # Forward pass
            outputs = model(**batch)
            loss = outputs.loss

            total_val_loss += loss.item()
            loss_list.append(loss.item())
            if (bi + 1) % 50 == 0:
                print(f"Progress [{bi+1:4d} / {num_batches}]")

    return total_val_loss, loss_list

def train(
        model,
        datasets,
        epochs: int = 3,
        batch_size: int = 8,
        learning_rate: float = 1e-5,
        warmup_steps: int = 100,
        weight_decay: float = 0.01,
        device: str = "cpu",
):
    train_dataset, val_dataset = datasets
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_dataloader = DataLoader(val_dataset, batch_size=batch_size)

    optimizer = AdamW(
        model.parameters(),
        lr=learning_rate,
        weight_decay=weight_decay,
    )

    lr_scheduler = None

    # lr_scheduler = get_scheduler(
    #     "linear",
    #     optimizer=optimizer,
    #     num_warmup_steps=warmup_steps,
    #     num_training_steps=epochs * len(train_dataloader),
    # )

    # # learning rate from 0.1 * 1.0 to 0.1 * 0.5 over 30 epochs
```

```
    # lr_scheduler = LinearLR(
    #     optimizer,
    #     start_factor=1.0, # Initial multiplier
    #     end_factor=0.5,   # Final multiplier
    #     total_iters=30    # Number of iterations for the change to occur
    # )

    train_losses = []
    val_losses = []
    val_perplexities = []
    for epoch in range(epochs):
        # Train step
        total_train_loss, train_loss_list = train_step(
            model,
            train_dataloader,
            optimizer,
            lr_scheduler,
            device,
        )

        # Average loss for the epoch
        avg_train_loss = total_train_loss / len(train_dataloader)
        train_losses.append(avg_train_loss)
        print(f"Epoch {epoch+1} Train Loss: {avg_train_loss:.6f}")
        print(f"Min train loss: [{np.min(train_loss_list):.6f}] | Max train loss: [{np.max(train_loss_list):.6f}] | Avg. train loss: [{np.mean(train_loss_list):.6f}] | Median train loss: [{np.median(train_loss_list)

        # Validation Step
        total_val_loss, val_loss_list = evaluate(
            model,
            val_dataloader,
            device,
        )

        avg_val_loss = total_val_loss / len(val_dataloader)
        val_perplexity = calculate_perplexity(avg_val_loss)
        val_losses.append(avg_val_loss)
        val_perplexities.append(val_perplexity)
        print(f"Epoch {epoch+1} Validation Loss: {avg_val_loss:.6f}, Perplexity: {val_perplexity:.6f}")
        print(f"Min val loss: [{np.min(val_loss_list):.6f}] | Max val loss: [{np.max(val_loss_list):.6f}] | Avg. val loss: [{np.mean(val_loss_list):.6f}] | Median val loss: [{np.median(val_loss_list):.6f}]")
        print()

    return train_losses, val_losses, val_perplexities
```

```
# Hyperparameters
epochs = 3
batch_size = 8
learning_rate = 2e-5
warmup_steps = 300
weight_decay = 0.01     # default = 0.01
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model.to(device)


print("Config:")
print(f"Epochs      : {epochs}")
print(f"Batch Size  : {batch_size}")
print(f"LR          : {learning_rate}")
# print(f"Warmup steps: {warmup_steps}")
# print(f"Warmup steps: 1 epoch")
print(f"Weight decay: {weight_decay}")
print(f"Device      : {device}")

train_losses, val_losses, val_perplexities = train(
    model,
    (train_dataset, val_dataset),
    epochs,
    batch_size,
    learning_rate,
    warmup_steps=warmup_steps,
    weight_decay=weight_decay,
```

```
    device=device,
)
```

```
Progress [ 600 / 1173]: Loss: 3.304991
Progress [ 700 / 1173]: Loss: 3.280263
Progress [ 800 / 1173]: Loss: 3.262101
Progress [ 900 / 1173]: Loss: 3.243654
Progress [1000 / 1173]: Loss: 3.226695
Progress [1100 / 1173]: Loss: 3.211684
Epoch 1 Train Loss: 3.203507
Min train loss: [2.597081] | Max train loss: [4.541179] | Avg. train loss: [3.203507] | Median train loss: [3.179168]
Progress [  50 / 292]
Progress [ 100 / 292]
Progress [ 150 / 292]
Progress [ 200 / 292]
Progress [ 250 / 292]
Epoch 1 Validation Loss: 2.898136, Perplexity: 18.140295
Min val loss: [2.356201] | Max val loss: [3.594781] | Avg. val loss: [2.898136] | Median val loss: [2.888747]

Progress [ 100 / 1173]: Loss: 2.963653
Progress [ 200 / 1173]: Loss: 2.968285
Progress [ 300 / 1173]: Loss: 2.974448
Progress [ 400 / 1173]: Loss: 2.977076
Progress [ 500 / 1173]: Loss: 2.968146
Progress [ 600 / 1173]: Loss: 2.967817
Progress [ 700 / 1173]: Loss: 2.965078
Progress [ 800 / 1173]: Loss: 2.959354
Progress [ 900 / 1173]: Loss: 2.955710
Progress [1000 / 1173]: Loss: 2.952070
Progress [1100 / 1173]: Loss: 2.948888
Epoch 2 Train Loss: 2.944697
Min train loss: [2.394552] | Max train loss: [3.456584] | Avg. train loss: [2.944697] | Median train loss: [2.941019]
Progress [  50 / 292]
Progress [ 100 / 292]
Progress [ 150 / 292]
Progress [ 200 / 292]
Progress [ 250 / 292]
Epoch 2 Validation Loss: 2.850985, Perplexity: 17.304820
Min val loss: [2.336570] | Max val loss: [3.556437] | Avg. val loss: [2.850985] | Median val loss: [2.842471]

Progress [ 100 / 1173]: Loss: 2.846951
Progress [ 200 / 1173]: Loss: 2.844765
Progress [ 300 / 1173]: Loss: 2.854484
Progress [ 400 / 1173]: Loss: 2.853493
Progress [ 500 / 1173]: Loss: 2.853271
Progress [ 600 / 1173]: Loss: 2.849185
Progress [ 700 / 1173]: Loss: 2.847358
Progress [ 800 / 1173]: Loss: 2.840188
Progress [ 900 / 1173]: Loss: 2.836843
Progress [1000 / 1173]: Loss: 2.833509
Progress [1100 / 1173]: Loss: 2.833067
Epoch 3 Train Loss: 2.834582
Min train loss: [2.391279] | Max train loss: [3.381976] | Avg. train loss: [2.834582] | Median train loss: [2.832157]
Progress [  50 / 292]
Progress [ 100 / 292]
Progress [ 150 / 292]
Progress [ 200 / 292]
Progress [ 250 / 292]
Epoch 3 Validation Loss: 2.857354, Perplexity: 17.415386
Min val loss: [2.334294] | Max val loss: [3.572143] | Avg. val loss: [2.857354] | Median val loss: [2.844999]
```

```
test_dataloader = DataLoader(test_dataset, batch_size=batch_size)
total_test_loss, test_loss_list = evaluate(model, test_dataloader, device)

avg_test_loss = total_test_loss / len(test_dataloader)
test_perplexity = calculate_perplexity(avg_test_loss)
print(f"Test Loss: {avg_test_loss:.6f}, Test Perplexity: {test_perplexity:.6f}")
```

```
Progress [  50 / 308]
Progress [ 100 / 308]
Progress [ 150 / 308]
Progress [ 200 / 308]
Progress [ 250 / 308]
Progress [ 300 / 308]
Test Loss: 3.766512, Test Perplexity: 43.229006
```

```python
import pandas as pd

train_metric = pd.DataFrame({
    "Epoch": list(range(epochs)),
    "Training Loss": train_losses,
    "Validation Loss": val_losses,
    "Validation Perplexity": val_perplexities,
})
train_metric
```

	Epoch	Training Loss	Validation Loss	Validation Perplexity
0	0	3.203507	2.898136	18.140295
1	1	2.944697	2.850985	17.304820
2	2	2.834582	2.857354	17.415386

```python
print(f"Avg. Test loss : {avg_test_loss:.6f}")
print(f"Test perplexity: {test_perplexity:.6f}")
```

```
Avg. Test loss : 3.766512
Test perplexity: 43.229006
```

Step 4: Submit your code and PDF

See the instruction in `hw0/hw0.md`